Recognizing Header Unit Imports Requires Full Preprocessing

Document:	P1703R0
Audience:	SG2, EWG

Authors:	Boris Kolpackov (Code Synthesis)
Reply-To:	boris@codesynthesis.com

Date:	2019-06-17

Abstract

Currently, recognizing header unit imports requires full preprocessing which is problematic for dependency scanning and partial preprocessing. This paper proposes changes that will allow handling such imports with the same degree of preprocessing as `#include` *directives.*

Contents

1 Background
2 Proposal
3 Before/After Tables ("Tony Tables")
  - 3.1 Affected Use Cases
  - 3.2 Unaffected Use Cases
  - 3.3 Unsupported Use Cases
4 Discussion
  - 4.1 Context-Sensitive Keywords
  - 4.2 One Line Requirement

5 Questions and Answers
5.1 Who will be in Cologne to present this paper?
5.2 Is there implementation experience?
5.3 Is there usage experience?
5.4 What shipping vehicle do you target with this proposal?
6 Acknowledgments

1 Background

With the current wording, recognizing a header unit `import` declaration requires performing macros replacement and tokenization of every line in a translation unit. As a representative example, consider the following line:

```
MYDECL import <int>;
```

Whether this is a header unit importation or something else depends on what `MYDECL` expands to. Compare:

```
#define MYDECL int x;

MYDECL import <int>;
```

And:

```
template <typename> class import;
#define MYDECL using x =

MYDECL import <int>;
```

> While the second example is contrived, it is valid (again, according to the current wording) because `import` is a context-sensitive keyword.

Requiring such full macro replacement is at a minimum wasteful for header dependency scanning but also may not be something that tools other than compilers can easily and correctly do.

Additionally, several implementations provide support for partial preprocessing (GCC's `-fdirectives-only` and Clang's `-frewrite-includes`) and this requirement is in conflict with the essence of that functionality.

More specifically, GCC is currently unable to support header unit imports in its `-M` (dependency scanning) and `-fdirectives-only` (partial preprocessing) modes because in these modes it does not perform macro replacement in non-directive lines.

While Clang currently performs full preprocessing in its `-M` and `-frewrite-includes` modes, there is agreement that it's not ideal for it to be impossible to correctly extract dependencies without full preprocessing.

Finally, consulting with the developers of `clang-scan-deps` (a Clang-based tool for fast dependency extraction) revealed that this requirement would be problematic for their implementation.

2 Proposal

We propose to further restrict header unit import declarations so that they can be recognized and handled with the same degree of preprocessing as `#include` directives.

Specifically, we propose recognizing a declaration as a header unit import if, additionally to restrictions in [cpp.module.1]:

1. It starts with the `import` token or `export import` token sequence that have not been produced by macro replacement.
2. Followed, after macro replacement, by *header-name-tokens*.
3. The entire, single, and only declaration is on one line.

We believe this should not detract much from usability because header imports are replacing `#include` directives where we have the same restrictions.

3 Before/After Tables ("Tony Tables")

3.1 Affected Use Cases

before	after

`int x; import <map>; int y;`	`int x;`
	`import <map>;`
	`int y;`

before	after
`import <map>; import <set>;`	`import <map>;` `import <set>;`

before	after

`export`
`import`
`<map>;`
	`export import <map>;`

before	after

`#ifdef MAYBE_EXPORT`
`export`
`#endif`
`import <map>;`
	`#ifdef MAYBE_EXPORT`
	`export import <map>;`
	`#else`
	`import <map>;`
	`#endif`

before	after

```
#define MAYBE_EXPORT export
MAYBE_EXPORT import <map>;
```

```
#define MAYBE_EXPORT
#ifdef MAYBE_EXPORT
export import <map>;
#else
import <map>;
#endif
```

3.2 Unaffected Use Cases

Header unit names are still macro-expanded (similar to `#include`):

```
#define MYMODULE <map>
import MYMODULE;
```

Normal module imports are unaffected:

```
import std.set; using int_set = std::set<int>;
```

3.3 Unsupported Use Cases

With the proposed change the following will no longer be possible:

```
#define MYIMPORT(x) import x
MYIMPORT(<set>);
```

Note also that the following is already impossible (because neither `#include` nor `import`'s closing `;` can be the result of macro replacement):

```
#define IMPORT_OR_INCLUDE(x) ???
IMPORT_OR_INCLUDE(<set>)
```

4 Discussion

4.1 Context-Sensitive Keywords

The proposed change does not fit well with the context-sensitive modules keywords semantics. In the current wording, the context is "wide" taking into account (after macro expansion) previous lines as well as `{}`-nesting. The following examples illustrate the problem:

```
#define MYDECL using x =
MYDECL
import <int>;
```

```
BEGIN_NAMESPACE

template<> class
import<int>;

END_NAMESPACE
```

Our proposed resolution is to adjust context-sensitivity for header unit imports to be based solely on the declaration itself. The fact that import should be at the beginning of the line followed by *header-name-tokens* and terminated with `;` already makes the "pattern" fairly constrained. We could not think of any plausible use-cases for `"` while `<` all seem to boil down to multi-line template-related declarations. And all such cases are easily fixed either by adjusting newlines or with `::`-qualification. For example:

before	after
`using x = import<int>;`	`using x = ::import<int>;`

```
template<> class
import<int>;
```

```
template<>
class import<int>;
```

> Doing a search for `import <` on https://codesearch.isocpp.org yielded 2562 matches which unfortunately also included `#import <...` directives. Doing a search for `#import <` produced 2540 matches. From this we can conclude (though, without seeing the actual code, with low degree of certainty), that there are 20 occurrences of the `import <` token sequence, however, not necessarily at the beginning of the line. We've managed to track at least some of these 20 matches to the Boost.Metaparse library with none of the occurrences being problematic.

4.2 One Line Requirement

Requiring the entire header unit `import` declaration to be on a single line is not strictly necessary. The benefit of this restriction is the simplification of tools that may then be able to reuse the same code to handle both `#include` directives and header unit `import` declarations (at least we found this to be the case for GCC). However, the ability to split the declaration across multiple lines could be beneficial in the presence of attributes. For example (courtesy of Richard Smith):

```
import "foo.h"
  [[clang::import_macros(FOO, BAR, BAZ, QUUX),
    clang::wrap_in_namespace(foo_namespace)]];
```

5 Questions and Answers

5.1 Who will be in Cologne to present this paper?

Boris Kolpackov

5.2 Is there implementation experience?

Yes, an implementation is available in the `boris/c++-modules-ex` GCC branch. This includes working `-fdirectives-only` mode.

One encouraging result of implementing the proposed change was the relative ease of generalizing the `#include` directive handling code in the GCC preprocessor (`libcpp`) and module mapper to also handle header unit imports.

5.3 Is there usage experience?

Yes, the `build2` build system implements support for header unit importation relying on this functionality.

5.4 What shipping vehicle do you target with this proposal?

The same as C++ Modules, presumably C++20.

6 Acknowledgments

To our knowledge this issue was first discovered and documented (in the GCC manual) by Nathan Sidwell.

Thanks to Nathan Sidwell, Richard Smith, Gabriel Dos Reis, Alex Lorenz, Michael Spencer, Cameron DaCamara, David Stone, and Ben Boeckel for discussions regarding this issue and for feedback on earlier drafts of this paper.